Отчёт по лабораторной работе №4

Реализация основных моделей в агентном подходе

Вакутайпа Милдред

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Модель 9
4.2 Сканирование коэффициента заразности 9
4.3 Исследование эффекта миграции 10
4.4 Многокритериальная оптимизация параметров 11
4.5 Сводная визуализация результатов 11
4.6 Дополнительные задания 12

5 Выводы **24**

Список литературы **25**

Список иллюстраций

4.1 Установка DrWatson 8
4.2 Создание каталога для проекта 8
4.3 график динамики SIR 9
4.4 Влияние изменения базовой зврвзности на эпидемические показатели 10
4.5 График эффекта миграции 11
4.6 Оптимальные параметры 11
4.7 результаты работ 12
4.8 Репродуктивное число 13
4.9 Миниальное значение ⊠ 14
4.10 Эффект гетерогенности при ⊠=(0.0, 0.0, 0.1) 14
4.11 графики эффекта гетерогенности при ⊠=(0.0, 0.0, 0.1) города A 14
4.12 графики эффекта гетерогенности при ⊠=(0.0, 0.0, 0.1)города B 15
4.13 графики эффекта гетерогенности при ⊠=(0.0, 0.1, 0.1)города C 15
4.14 Эффект гетерогенности при ⊠=(0.03, 0.1, 0.4) 16
4.15 График эффекта гетерогенности при ⊠=(0.03, 0.1, 0.4) города A 16
4.16 График эффекта гетерогенности при ⊠=(0.03, 0.1, 0.4) города B 17
4.17 График эффекта гетерогенности при ⊠=(0.03, 0.1, 0.4) города C 17
4.18 Эффект гетерогенности при ⊠=(0.03, 0.6, 0.4) 18
4.19 График эффекта гетерогенности при ⊠=(0.03, 0.6, 0.4) города A 18
4.20 График эффекта гетерогенности при ⊠=(0.03, 0.6, 0.4) города B 18
4.21 График эффекта гетерогенности при ⊠=(0.03, 0.6, 0.4) города C 19
4.22 Эффект карантина 20
4.23 Нахождение опитимальных параметров 23
4.24 Применение опитимальных параметров 23

Список таблиц

1 Цель работы

Цель данной работы – реализовать основных моделей в агентном подходе к имитационному моделированию.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль. Сгенерировать из литературного кода:
    - чистый код;
    - jupyter notebook;
    - документацию в формате Quarto.
5. Выполнить код из jupyter notebook.
6. Интегрировать документацию в формате Quarto в отчёт.
7. Ответить на дополнительные задания

3 Теоретическое введение

Создадим агентную модель распространения инфекционного заболевания на основе классической компартментальной модели SIR (Susceptible-Infectious-Recovered). Модель будет реализована с использованием пакета Agents.jl. В отличие от классической модели на дифференциальных уравнениях, агентный подход позволит учесть индивидуальные характеристики, пространственную структуру и стохастичность процессов.

4 Выполнение лабораторной работы

Используя программмного обеспечения julia и пакт Pkg, я установила DrWatson(рис. 4.1).

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.10.10 (2025-06-27)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org/ release
|__/                   |

julia> using Pkg

julia> Pkg.add("DrWatson")
    Updating registry at `~/.julia/registries/General.toml`
```

Рисунок 4.1: Установка DrWatson

Далее я создала каталог для работы project и в нем установила остальные пакты, которые мне нужно будет (рис. 4.2)

```
julia> initialize_project("project"; authors="mildred", git=false)
  Activating new project at `~/work/study/2026-1/2026-1--study--simmo
n-modelling/labs/lab04/project`
   Resolving package versions...
    Updating `~/work/study/2026-1/2026-1--study--simmod/2026-1--study
s/lab04/project/Project.toml`
  [634d3b9d] + DrWatson v2.19.1
    Updating `~/work/study/2026-1/2026-1--study--simmod/2026-1--study
s/lab04/project/Manifest.toml`
  [0b6fb165] + ChunkCodecCore v1.0.1
  [4c0bbee4] + ChunkCodecLibZlib v1.0.0
  [55437552] + ChunkCodecLibZstd v1.0.0
  [634d3b9d] + DrWatson v2.19.1
```

Рисунок 4.2: Создание каталога для проекта

4.1 Модель

Я выполнила предложенный код для модели src/sir_model.jl. В нем описывается агенты, пространство состояния и другие параметры модели. На каждом шаге (соответствует одному дню) для каждого агента выполняются миграцию, передачу инфекции, обновление счетчика, выздоровление или смерть.

Далее я выполнила предлоденный код базового эксперимента. Запускает один эксперимент с фиксированными параметрами (по умолчанию) и сохраняет динамику численности агентов. Это служит для проверки работоспособности модели и получения базового понимания эпидемического процесса. (рис. 4.3)

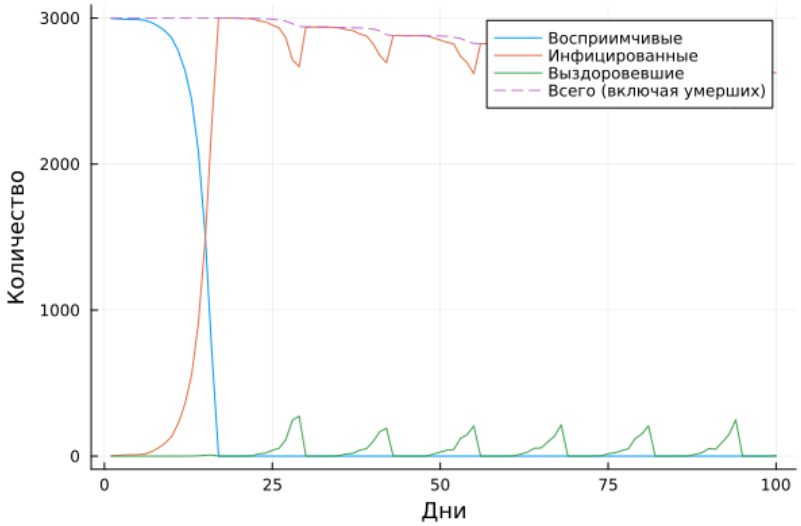

Рисунок 4.3: график динамики SIR

4.2 Сканирование коэффициента заразности

Я выполнила скрипт sir_scan_beta.jl, который исследует, как изменение базовой заразности (β_und и пропорционально β_det) влияет на эпидемические показатели: пик заболеваемости, долю переболевших, число умерших. Выполняется параметрическое сканирование с несколькими повторными прогонами для учёта стохастичности. В ходе этого получила датафрейм результатов и следующий

график (рис. 4.4) .

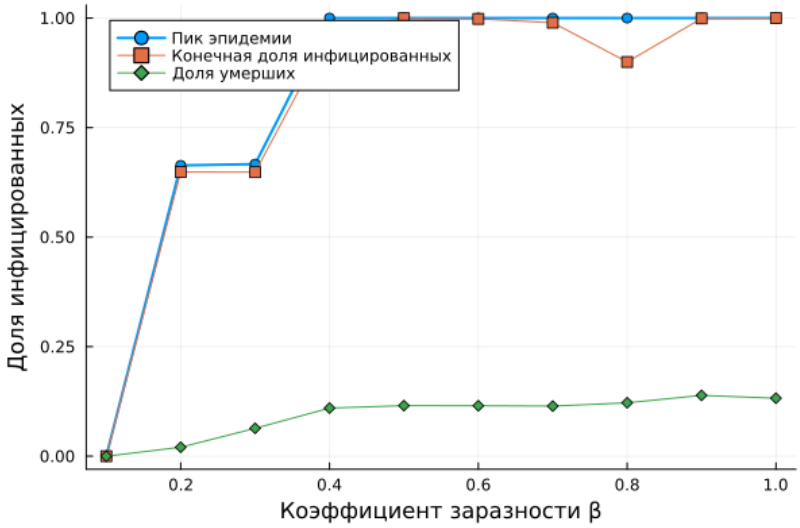

Рисунок 4.4: Влияние изменения базовой зврвзности на эпидемические показатели

4.3 Исследование эффекта миграции

Я выполнила предложенный код для исследовавния эффекта миграции scripts/sir_migration_effect.jl, который исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии (время достижения пика) и масштаб пика. Инфекция начинается только в одном городе, остальные изначально здоровы. В ответе получила следующий график (рис. 4.5):

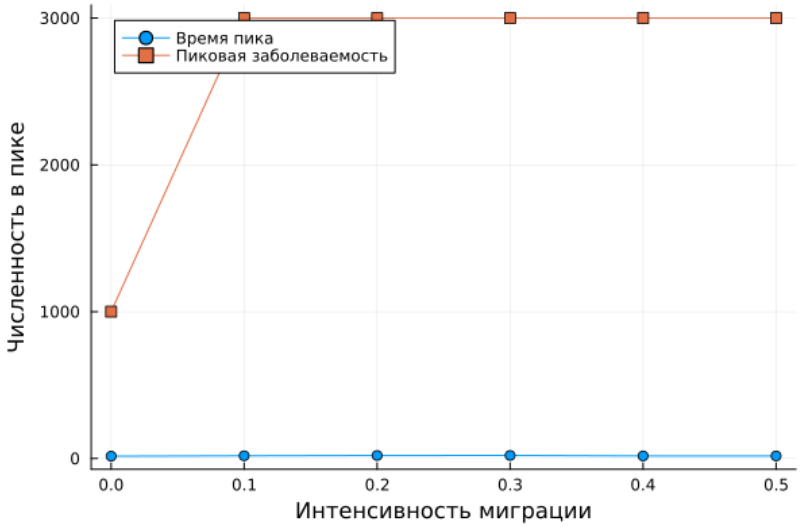

Рисунок 4.5: График эффекта миграции

4.4 Многокритериальная оптимизация параметров

Выполнила код для многокритериальной оптимизации параметров scripts/sir_optimize_parameters.jl. Он ищет оптимальные комбинации параметров, минимизирующие одновременно два критерия: пиковую заболеваемость и долю умерших. Использует эволюционный алгоритм (Borg MOEA) из пакета BlackBoxOptim. В ответе получила следующий (рис. 4.6):

```
Оптимальные параметры:
β_und = 0.1098141406037895
Время выявления = 5 дней
Смертность = 0.04273969687980218
Достигнутые показатели:
Пик заболеваемости: 0.0006666666666666666
Доля умерших: 0.0
```

Рисунок 4.6: Оптимальные параметры

4.5 Сводная визуализация результатов

Выполнила предложенный код для визуализации результатов (рис. 4.7). Он загружает CSV с помощью CSV.read и преобразует в DataFrame. При необходимости

усредняет данные по повторным прогонам (группировка по β_und). В приведённой реализации используется прямое построение без усреднения, но можно добавить. Создаёт фигуру с тремя вертикальными подграфиками:

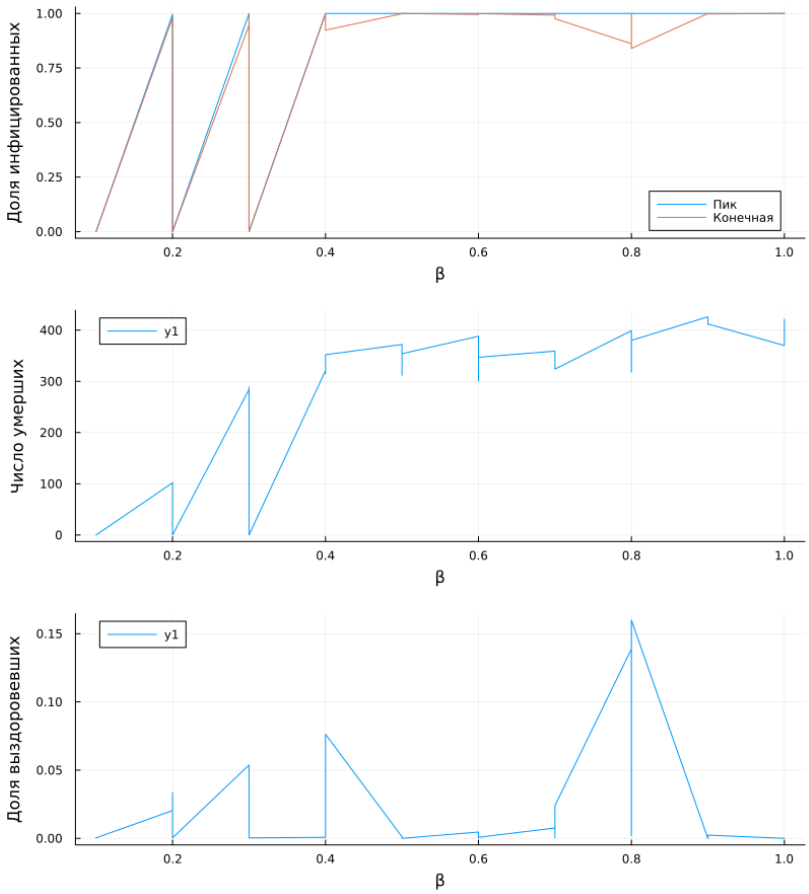

Рисунок 4.7: результаты работ

4.6 Дополнительные задания

1. Я запустила модель с параметрами по умолчанию и построила график динамики численности ⊠ , ⊠ , ⊠ (рис. 4.3). Чтобы определить базовое репродуктивное число R_0 я добавила в коде следующие строка:

```
y = 1/params[:infection_period]
B = params[:β_und][1]
R_0 = B/y

println("базовое репродуктивное число: ", R_0)
```

В ответе получила (рис. 4.8).

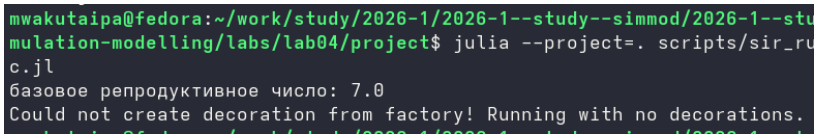

Рисунок 4.8: Репродуктивное число

2. Чтобы найти минимальное значение β, при котором возникает эпидемия, я добавила следующие строка в скрипте sir_scan_beta:

```
epidemic_threshold = nothing

for row in eachrow(grouped)
    if row.mean_peak > 0.05
        epidemic_threshold = row.beta
        break
    end
end
```

Она равна 0.21 и больше чем теоретический порог в 2 раза (рис. 4.9)

```
Завершен эксперимент с beta = 0.71, seed = 44
Завершён эксперимент с beta = 0.81, seed = 42
Завершён эксперимент с beta = 0.81, seed = 43
Завершён эксперимент с beta = 0.81, seed = 44
Завершён эксперимент с beta = 0.91, seed = 42
Завершён эксперимент с beta = 0.91, seed = 43
Завершён эксперимент с beta = 0.91, seed = 44
минимальная β для эпидемии (>5%): 0.21
R₀: 2.94
```

Рисунок 4.9: Миниальное значение ⊠

3. Я создала новый скрипт scripts/sir_diff_cities и исползовала три разные значения параметр ⊠ для всех городов. Сначала я задала значения 0.0, 0.0, 0.1 и получила в ответе (рис. 4.10)

```
  Конечная доля инфицированных: 0 (0.0%)
  Город A, пик: 0 (0.0%)
  Город B, пик: 0 (0.0%)
  Город C, пик: 1 (0.1%)
Could not create decoration from factory! Running w
РЕЗУЛЬТАТЫ СОХРАНЕНЫ
mwakutaipa@fedora:~/work/study/2026-1/2026-1--study
/labs/lab04/project$ julia --project=. scripts/sir_
```

Рисунок 4.10: Эффект гетерогенности при ⊠=(0.0, 0.0, 0.1)

Я также построила их графики:

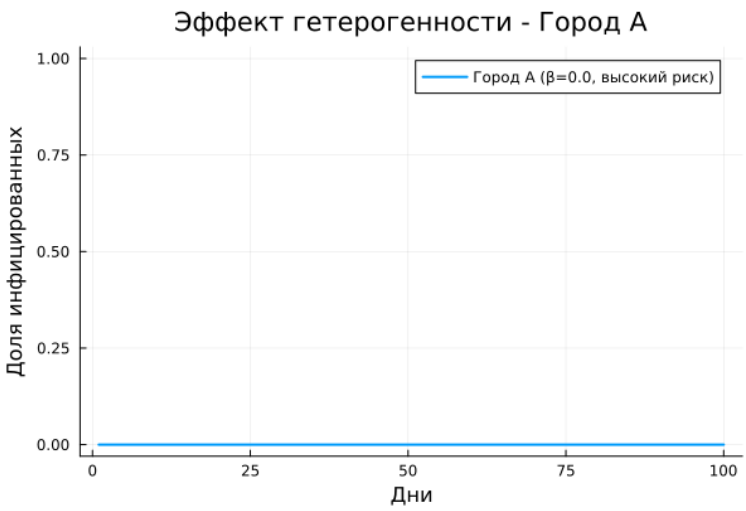

Рисунок 4.11: графики эффекта гетерогенности при ⊠=(0.0, 0.0, 0.1) города A

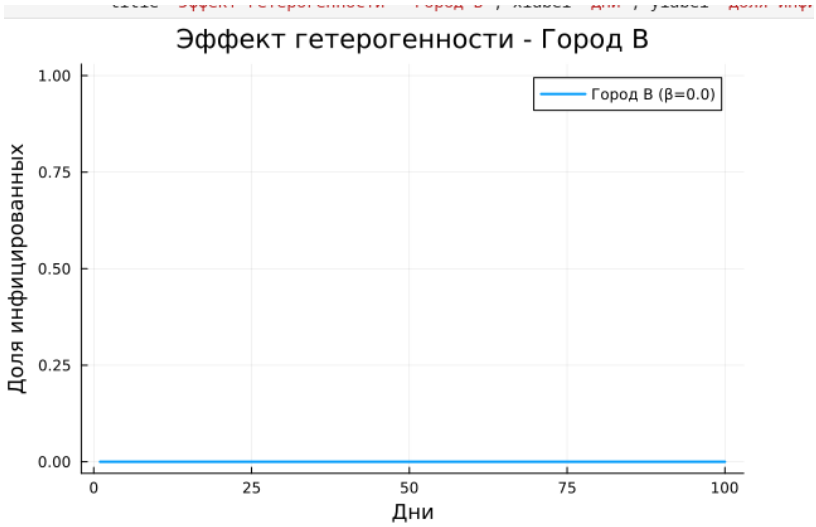

Рисунок 4.12: графики эффекта гетерогенности при ⊠=(0.0, 0.0, 0.1)города B

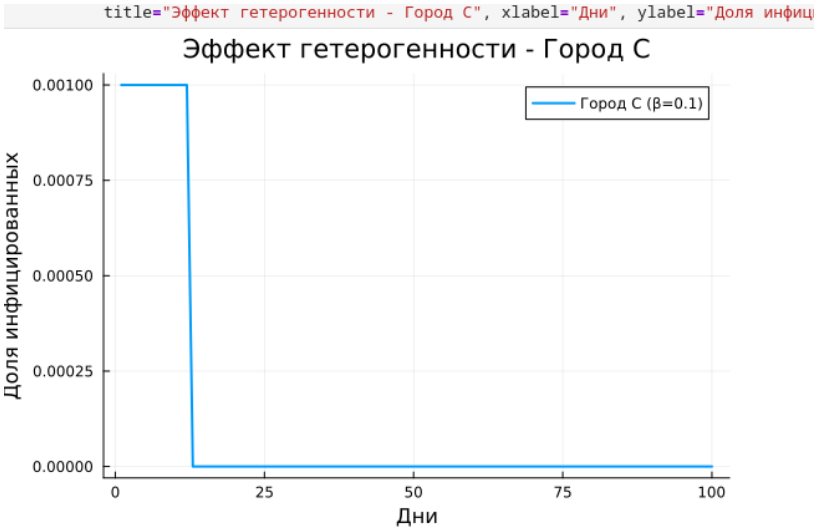

Рисунок 4.13: графики эффекта гетерогенности при ⊠=(0.0, 0.1, 0.1)города C

во вторых потестировала с ззначениями 0.03, 0.1 и 0.4 которые меньше, чем минимальная ⊠ в первых двух городах и получила в ответе (рис. 4.14)

```
Однородный сценарий (β = [0.5, 0.5, 0.5]):
  Пик заболеваемости: 1000 (33.3%)
  Конечная доля инфицированных: 2625 (87.5%)

Гетерогенный сценарий (β = [0.03, 0.1, 0.4]):
  Пик заболеваемости: 431 (14.4%)
  Конечная доля инфицированных: 1279 (42.6%)
  Город A, пик: 422 (42.2%)
  Город B, пик: 426 (42.6%)
  Город C, пик: 431 (43.1%)
```

Рисунок 4.14: Эффект гетерогенности при ⊠=(0.03, 0.1, 0.4)

Графики:

Рисунок 4.15: График эффекта гетерогенности при ⊠=(0.03, 0.1, 0.4) города A

Рисунок 4.16: График эффекта гетерогенности при ⊠=(0.03, 0.1, 0.4) города B

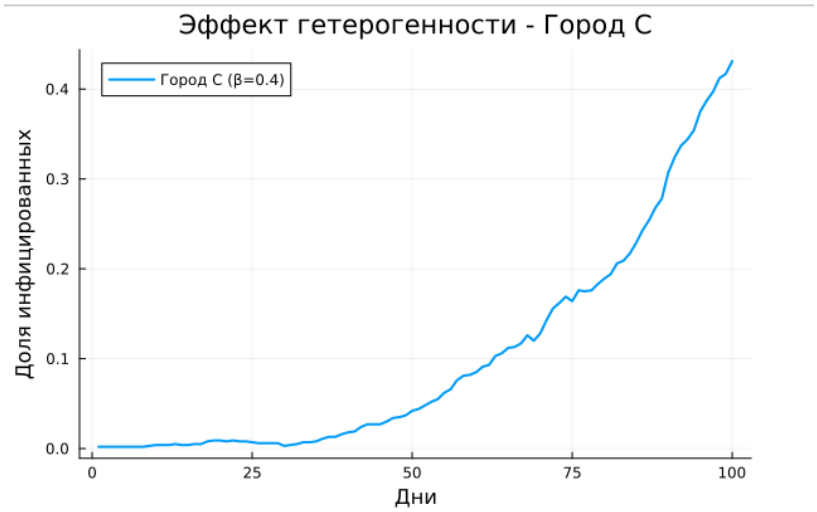

Рисунок 4.17: График эффекта гетерогенности при ⊠=(0.03, 0.1, 0.4) города C

в последнем варианте я повысила ⊠ во втором городе в 6 раз и получила в ответе (рис. 4.18)

```
Гетерогенный сценарий (β = [0.03, 0.6, 0.4]):
  Пик заболеваемости: 998 (33.3%)
  Конечная доля инфицированных: 2636 (87.9%)
  Город A, пик: 996 (99.6%)
  Город B, пик: 998 (99.8%)
  Город C, пик: 995 (99.5%)
Could not create decoration from factory! Running wit
```

Рисунок 4.18: Эффект гетерогенности при ⊠=(0.03, 0.6, 0.4)

Графики получились похоже исходных графиков:

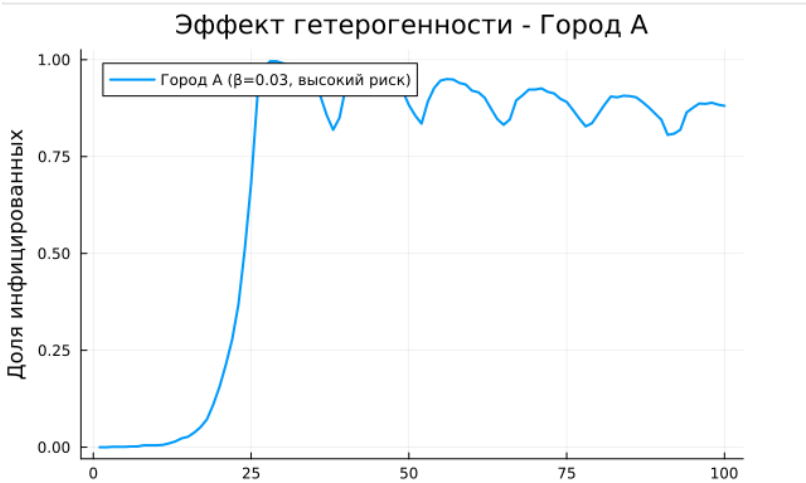

Рисунок 4.19: График эффекта гетерогенности при ⊠=(0.03, 0.6, 0.4) города A

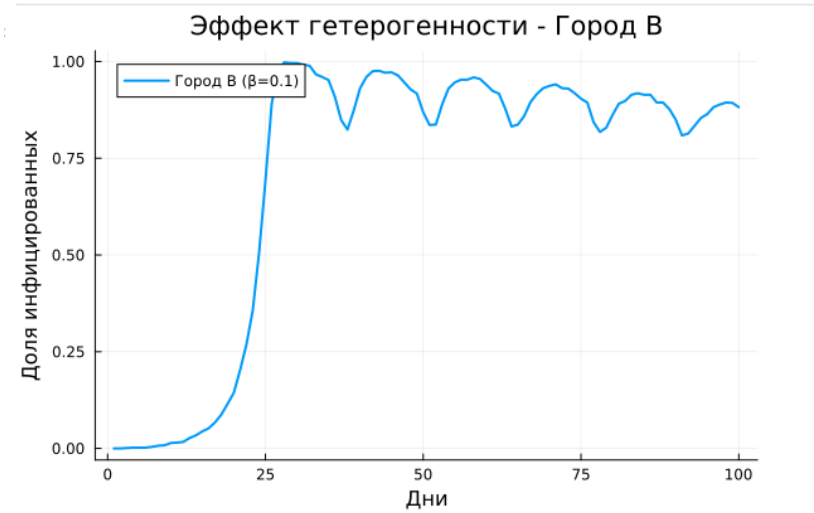

Рисунок 4.20: График эффекта гетерогенности при ⊠=(0.03, 0.6, 0.4) города B

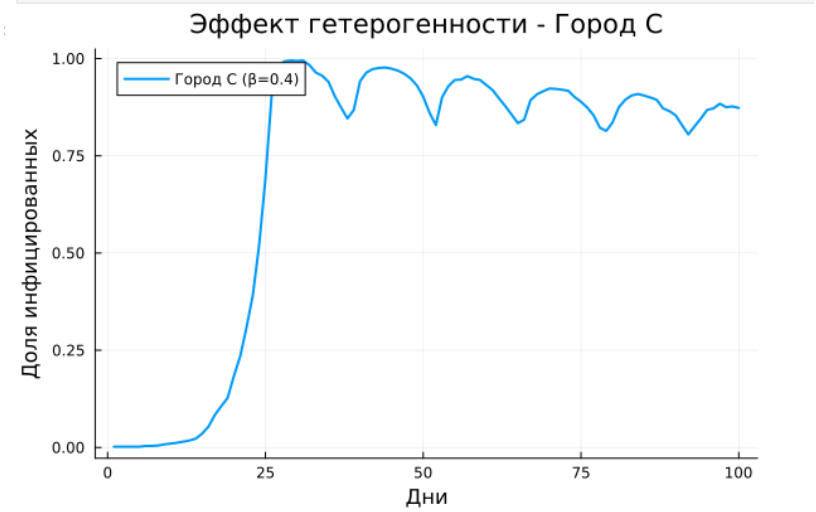

Рисунок 4.21: График эффекта гетерогенности при ⊠=(0.03, 0.6, 0.4) города C

4. Запускла скрипт sir_migration_effect и проверила созданный датасет чтобы узнать при какой интенсивности миграции наблюдается минимальное время до пика и заметила ,что при интенсивности 0.0 время до пика минимально.

5. Я добавила возможность закрытия города (обнуление миграции из него) при превышении порога заболеваемости:

```
thresholds = [0.0, 0.05, 0.10, 0.20]  # 0 = ⊠⊠⊠ ⊠⊠⊠⊠⊠⊠⊠⊠⊠
scenario_names = ["⊠⊠⊠ ⊠⊠⊠⊠⊠⊠⊠⊠⊠", "⊠⊠⊠⊠⊠⊠ (5%)", "⊠⊠⊠⊠⊠⊠⊠⊠⊠ (10%)", "⊠⊠⊠⊠

results_q = []
for (i, thresh) in enumerate(thresholds)
    println("\n⊠⊠⊠⊠⊠⊠⊠⊠⊠⊠⊠: $(scenario_names[i])")
    result = run_quarantine_scenario(thresh)
    push!(results_q, (scenario=scenario_names[i],
                      peak=round(result.peak*100, digits=1),
                      deaths=result.deaths))
end
```

Получила ,что количество умерших уменьшилось но не сильно (рис. 4.22):

```
=== Эффективность карантина ===
4×3 DataFrame
 Row │ scenario          peak     deaths
     │ String            Float64  Int64
─────┼──────────────────────────────────
   1 │ Без карантина      100.0     584
   2 │ Ранний (5%)        100.0     540
   3 │ Умеренный (10%)    100.0     543
   4 │ Поздний (20%)      100.0     524
```

Рисунок 4.22: Эффект карантина

6. Использовала оптимизацию, чтобы найти параметры, которые минимизируют общее число умерших при сохранении пика заболеваемости ниже 30%.

```
function cost_multi_constrained(x)
    β_und = x[1]
    detection_time = round(Int, x[2])
    death_rate = x[3]

    replicates = 5
    peak_vals = Float64[]
    dead_vals = Int[]

    for rep in 1:replicates
        model = initialize_sir(;
            Ns = [1000, 1000, 1000],
            β_und = fill(β_und, 3),
            β_det = fill(β_und/10, 3),
            infection_period = 14,
            detection_time = detection_time,
            death_rate = death_rate,
            reinfection_probability = 0.1,
```

```
            Is = [0, 0, 1],
            seed = 42 + rep,
            n_steps = 100,
        )

        peak = 0.0
        for step in 1:100
            Agents.step!(model, 1)
            frac = infected_count(model) / 3000
            if frac > peak
                peak = frac
            end
        end

        deaths = 3000 - nagents(model)

        if peak > 0.3
            peak = peak + 10.0
        end

        push!(peak_vals, peak)
        push!(dead_vals, deaths)
    end

    return (mean(dead_vals), mean(peak_vals))
end

result_constrained = bboptimize(
```

```
    cost_multi_constrained,
    Method = :borg_moea,
    FitnessScheme = ParetoFitnessScheme{2}(is_minimizing = true),
    SearchRange = [
        (0.1, 1.0),
        (3.0, 14.0),
        (0.01, 0.1),
    ],
    NumDimensions = 3,
    MaxTime = 120,
    TraceMode = :compact,
)

best_constrained = best_candidate(result_constrained)
fitness_constrained = best_fitness(result_constrained)

println("\n□□□□□□□□□□□ □□□□□□□□□ (□ □□□□□□□□□□□□ □□□ < 30%):")
println("□_und = $(round(best_constrained[1], digits=3))")
println("□□□□□ □□□□□□□□□ = $(round(Int, best_constrained[2])) □□□□")
println("□□□□□□□□□□□ □□□□□□□□□□ = $(round(best_constrained[3], digits=4)
println("\n□□□□□□□□□□□ □□□□□□□□□□:")
println("□□□□□□□ □□□□□ □□□□□□□ = $(round(fitness_constrained[1]))")
println("□□□□□□□ □□□ □□□□□□□□□□□□□ = $(round(fitness_constrained[2]*100,
```

```
Оптимальные параметры:
β_und = 0.12678235838688698
Время выявления = 4 дней
Смертность = 0.09837770804825781
Достигнутые показатели:
Пик заболеваемости: 0.0006666666666666666
Доля умерших: 6.666666666666667e-5
Starting optimization with optimizer BlackBoxOptim.BorgMOEA{EpsBoxDominanceFi
tnessScheme{2, Float64, true, typeof(sum)}, BlackBoxOptim.ProblemEvaluator{Tu
ple{Float64, Float64}, IndexedTupleFitness{2, Float64}, EpsBoxArchive{2, Floa
t64, EpsBoxDominanceFitnessScheme{2, Float64, true, typeof(sum)}}, FunctionBa
sedProblem{typeof(cost_multi_constrained), ParetoFitnessScheme{2, Float64, tr
ue, typeof(sum)}, ContinuousRectSearchSpace, Nothing}}, FitPopulation{Indexed
TupleFitness{2, Float64}}, FixedGeneticOperatorsMixture, RandomBound{Continuo
usRectSearchSpace}}
0.00 secs, 0 evals, 0 steps
```

Рисунок 4.23: Нахождение опитимальных параметров

```
Оптимальные параметры (с ограничением пик < 30%):
β_und = 0.124
Время выявления = 6 дней
Коэффициент смертности = 0.0152

Достигнутые показатели:
Среднее число умерших = 0.0
Средний пик заболеваемости = 0.0%
```

Рисунок 4.24: Применение опитимальных параметров

5 Выводы

При выполнении данной работы я реализовала основных моделей в агентном подходе к имитационному моделированию.

Список литературы